

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I A Raksha 192311421 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Raksha

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

Problem Statement

Warehouses that rely on manual, spreadsheet-based stock tracking suffer from inaccurate stock counts, delayed reordering, misplaced items, and slow order fulfillment. The assigned project, the Smart Warehouse Inventory Management Platform, is a full-stack web application that digitizes warehouse operations end-to-end: real-time stock tracking via barcode/RFID scanning, automated low-stock alerts, purchase-order and shipment management, warehouse bin/zone mapping, and an analytics dashboard for managers. The platform is built with a React frontend, a Node.js/Express REST API backend, and a MongoDB database, and is containerized with Docker so it can be developed, tested, and deployed consistently across environments.

Project Objectives

- Provide real-time, centralized visibility of inventory quantities and locations across the warehouse.
- Automate stock replenishment by triggering alerts when items fall below a reorder threshold.
- Digitize inbound (receiving) and outbound (picking, packing, shipping) workflows.
- Support role-based access for staff, procurement, logistics, and administrators.
- Use Git for collaborative, versioned development and Docker for consistent, portable deployment.

Functional Requirements

- FR1 — Users can authenticate and are granted permissions based on their assigned role.
- FR2 — Staff can scan an item (barcode/RFID) to update stock quantity and bin location in real time.
- FR3 — The system automatically raises a low-stock alert when a SKU crosses its reorder threshold.
- FR4 — Procurement can create and track purchase orders to suppliers.
- FR5 — Logistics staff can generate picking lists and update shipment status for outbound orders.
- FR6 — Managers can view a live dashboard of stock levels, turnover, and warehouse performance.
- FR7 — Administrators can configure warehouse zones, aisles, and bins.

Expected Outcomes

- A fully containerized three-tier application (frontend, backend, database) deployable with a single command.
- A Git repository with a clean branching history demonstrating collaborative development practices.
- Reduced stock discrepancies and faster order turnaround compared to manual tracking.
- A reproducible development-to-production pipeline enabled by Docker and Docker Compose.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

Repository Folder Structure

The repository follows a monorepo structure that separates the frontend, backend, and supporting documentation into independently buildable units, each with its own Dockerfile, while sharing a single Git history and a top-level docker-compose.yml that orchestrates them together.

Project Repository Folder Structure

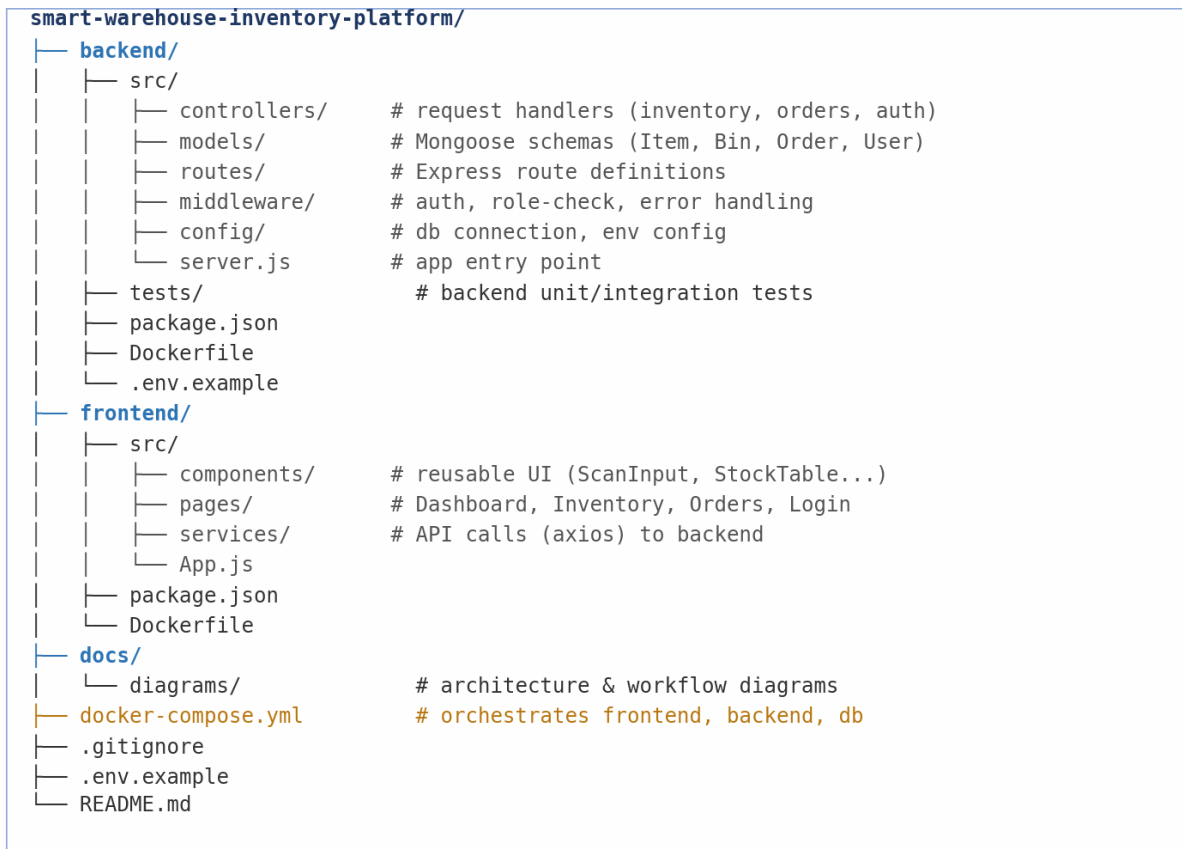


Figure 1 — Repository folder structure for the Smart Warehouse Inventory Management Platform

Purpose of Major Folders and Files

Folder / File	Purpose
backend/src/controllers/	Request handlers containing business logic for inventory, orders, and authentication.
backend/src/models/	Mongoose schemas defining Item, Bin, Order, and User data structures.
backend/src/routes/	Express route definitions mapping HTTP endpoints to controllers.
backend/src/middleware/	Reusable middleware for JWT authentication, role checks, and centralized error handling.
backend/src/config/	Database connection and environment configuration.
backend/tests/	Unit and integration tests for backend logic (Jest/Mocha).
backend/Dockerfile	Instructions to build the backend API container image.
frontend/src/components/	Reusable UI components (e.g., ScanInput, StockTable).
frontend/src/pages/	Top-level pages such as Dashboard, Inventory, Orders, and Login.
frontend/src/services/	Axios-based API client modules that call the backend REST API.
frontend/Dockerfile	Instructions to build and serve the production React bundle via NGINX.
docker-compose.yml	Defines and orchestrates the frontend, backend, and database services together.

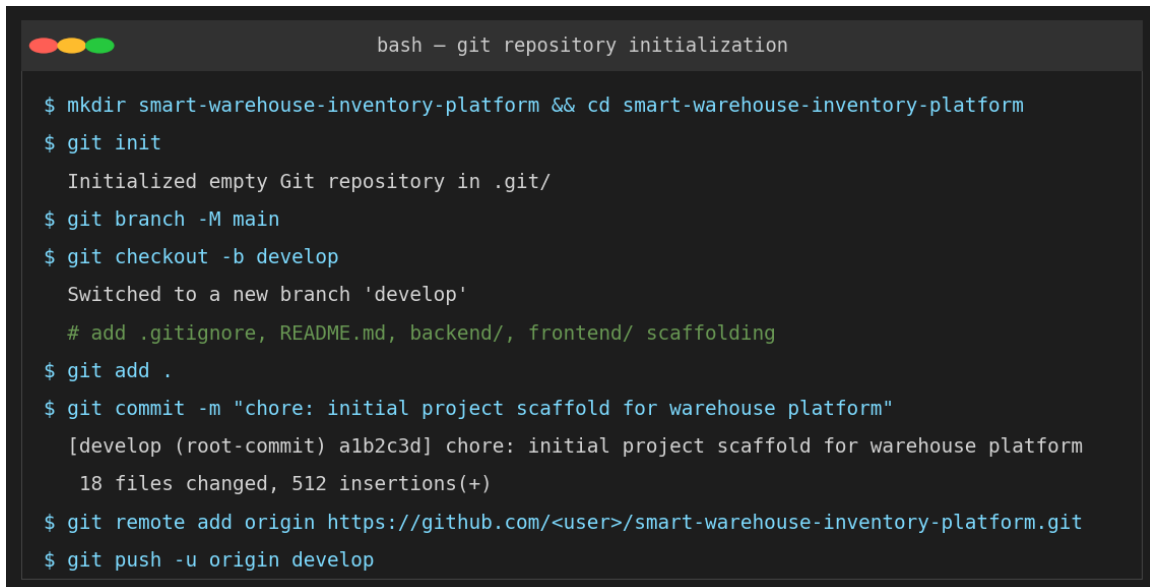
.gitignore	Excludes node_modules, build artifacts, and .env files from version control.
.env.example	Documents required environment variables without exposing real secrets.
docs/diagrams/	Architecture, branching, and workflow diagrams referenced in this report.
README.md	Project overview, setup instructions, and usage documentation.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

Repository Initialization Steps

- Create the project root folder and run git init to create the local .git repository.
- Rename the default branch to main and create a develop branch as the integration branch.
- Add the initial project scaffold (backend/, frontend/, .gitignore, README.md) and make the first commit.
- Create a remote repository on GitHub and link it with git remote add origin <url>.
- Push both main and develop branches upstream with git push -u origin <branch>.



```

bash – git repository initialization

$ mkdir smart-warehouse-inventory-platform && cd smart-warehouse-inventory-platform
$ git init
  Initialized empty Git repository in .git/
$ git branch -M main
$ git checkout -b develop
  Switched to a new branch 'develop'
# add .gitignore, README.md, backend/, frontend/ scaffolding
$ git add .
$ git commit -m "chore: initial project scaffold for warehouse platform"
[develop (root-commit) a1b2c3d] chore: initial project scaffold for warehouse platform
 18 files changed, 512 insertions(+)
$ git remote add origin https://github.com/<user>/smart-warehouse-inventory-platform.git
$ git push -u origin develop

```

Figure 2 — Terminal commands used to initialize the Git repository

Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by git init; stores the entire object database, refs, and history.
.gitignore	Lists files/folders (node_modules/, .env, dist/, build/) that Git should never track.
README.md	Documents project purpose, setup steps, and usage for collaborators.
.env.example	Template of required environment variables, committed

	safely without real secret values.
package.json / package-lock.json	Tracks Node.js dependencies and locks exact versions for reproducible installs.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

Selected Branching Model: Gitflow

The project uses a Gitflow-based branching strategy with five branch types: main, develop, feature/*, release/*, and hotfix/*.

Branch	Purpose
main	Always reflects the latest production-ready, deployed release. Tagged with version numbers (e.g., v1.0.0).
develop	Integration branch where completed features are merged and tested together before release.
feature/<name>	Short-lived branch for one feature or user story (e.g., feature/inventory-scan), branched from develop.
release/<version>	Cut from develop for final QA, version bumps, and documentation before merging to main.
hotfix/<name>	Branched from main to urgently patch a production bug, then merged back into both main and develop.

Git Branching Strategy — Smart Warehouse Inventory Management Platform

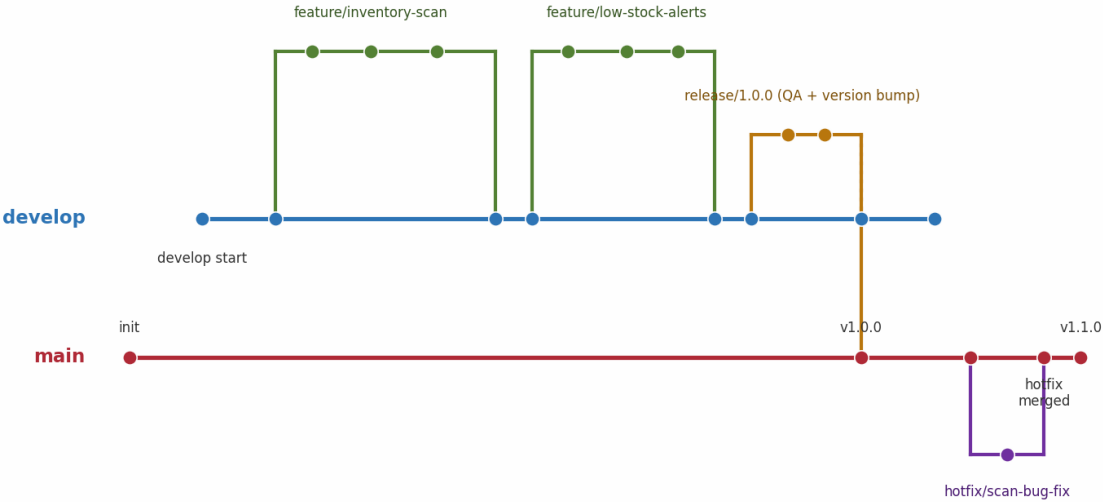


Figure 3 — Git branching strategy (Gitflow) for the warehouse platform

Why This Model Fits the Project

- Multiple independent modules (inventory scanning, alerts, orders, dashboard) map naturally to isolated feature branches, reducing merge conflicts between team members.
- The develop branch provides a stable integration point for continuous testing before anything reaches production.

- The release branch allows QA and documentation to happen without blocking new feature work on develop.
- The hotfix branch enables urgent production fixes (e.g., a scanning bug affecting live stock counts) without waiting for the next full release cycle.
- main always represents a deployable state, which pairs naturally with Docker image tagging — each merge to main can trigger a new versioned image build.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

Demonstrated Workflow

The example below shows a complete feature cycle: branching from develop, committing incremental changes, pushing, opening a pull request, and merging back with a conflict resolved along the way.

```

bash – feature branch workflow & conflict resolution

$ git checkout develop && git pull origin develop
$ git checkout -b feature/inventory-scan
  Switched to a new branch 'feature/inventory-scan'
  # implement barcode scan endpoint + UI
$ git add backend/src/controllers/inventoryController.js
$ git commit -m "feat(inventory): add barcode scan endpoint to update stock"
$ git add frontend/src/components/ScanInput.jsx
$ git commit -m "feat(ui): add ScanInput component for receiving workflow"
$ git push origin feature/inventory-scan
  # Pull Request opened: feature/inventory-scan -> develop
$ git checkout develop
$ git merge --no-ff feature/inventory-scan
  Auto-merging backend/src/routes/inventoryRoutes.js
  CONFLICT (content): Merge conflict in backend/src/routes/inventoryRoutes.js
  # resolve conflict manually in editor, keep both route additions
$ git add backend/src/routes/inventoryRoutes.js
$ git commit -m "merge: resolve route conflict, merge feature/inventory-scan into develop"
$ git push origin develop
  
```

Figure 4 — Feature branch workflow including conflict resolution



Figure 5 — End-to-end Git workflow from feature branch to release

Purpose of Each Git Operation

Operation	Purpose
git checkout -b feature/x	Creates and switches to a new isolated branch for a single feature.
git add	Stages specific changed files for the next commit.

git commit -m "..."	Records a snapshot of staged changes with a descriptive message.
git push origin <branch>	Uploads local commits to the remote repository (GitHub).
Pull Request	Requests review and discussion before merging into develop, enabling collaboration.
git merge --no-ff	Merges the feature branch into develop, preserving a merge commit for traceability.
Conflict resolution	Manually reconciles overlapping changes (e.g., same file edited on two branches) before completing the merge.
git pull	Fetches and merges remote changes into the current local branch to stay in sync with teammates.

6. Commit History Analysis

- Present the commit history of the project.
- Explain how commit messages follow good version control practices and contribute to project maintenance.

Project Commit History

```

bash - commit history

$ git log --oneline --graph --decorate -12
* f3a9c21 (HEAD -> develop) merge: resolve route conflict, merge feature/inventory-scan
|\
| * d8e21aa (feature/inventory-scan) feat(ui): add ScanInput component for receiving workflow
| * 9c4b7e0 feat(inventory): add barcode scan endpoint to update stock
|/
* 7b1f402 feat(auth): implement role-based JWT authentication middleware
* 4e9a10c feat(alerts): add low-stock threshold checker and notification job
* 2d5c8b1 feat(bins): add warehouse zone and bin configuration API
* c1a7f88 test(inventory): add unit tests for stock update logic
* a1b2c3d chore: initial project scaffold for warehouse platform

```

Figure 6 — Commit history graph (git log --oneline --graph --decorate)

Commit Message Conventions

All commits follow the Conventional Commits format: <type>(<scope>): <short description>, keeping the subject line under ~72 characters and written in the imperative mood.

Type	Used For	Example
feat	A new feature	feat(inventory): add barcode scan endpoint to update stock
fix	A bug fix	fix(auth): correct expired-token handling in middleware
chore	Tooling/scaffolding, no source logic change	chore: initial project scaffold for warehouse platform
test	Adding or updating tests	test(inventory): add unit tests for stock update logic
merge	Merge commits, often	merge: resolve route conflict, merge

	documenting conflict resolution	feature/inventory-scan into develop
--	---------------------------------	-------------------------------------

How This Contributes to Project Maintenance

- Each commit represents one logical, reviewable change, making git log and git blame useful for tracing why a line changed.
- Consistent prefixes (feat, fix, chore, test) allow automatic changelog generation and quick filtering of history.
- Small, frequent commits make git revert and git bisect effective for isolating the source of a regression.
- Descriptive messages reduce onboarding time for new team members reviewing project history.

7. Docker Environment Design

- Explain why Docker is suitable for the assigned project.
- Identify the application components that require containerization.

Why Docker Is Suitable for This Project

- The platform has three distinct runtimes (React/Node build tools, Node.js API, MongoDB) — Docker isolates each so their dependencies never conflict.
- Warehouse deployments may run on different physical or cloud servers; Docker guarantees the application behaves identically regardless of host OS.
- docker compose up allows the entire multi-service stack to be started with a single command, drastically simplifying onboarding and demos.
- Images can be tagged and pushed to a registry, giving traceable, rollback-capable deployments tied to Git release tags.
- Containers can be scaled independently — e.g., running multiple backend replicas behind a load balancer during peak warehouse activity.

Application Components Requiring Containerization

Component	Container	Reason
React frontend	warehouse-frontend	Built into static assets and served via NGINX; isolated from backend build tools.
Node.js/Express API	warehouse-backend	Runs the REST API; needs its own Node runtime and npm dependencies.
MongoDB	warehouse-mongo	Persists inventory, order, and user data; isolated with its own data volume.

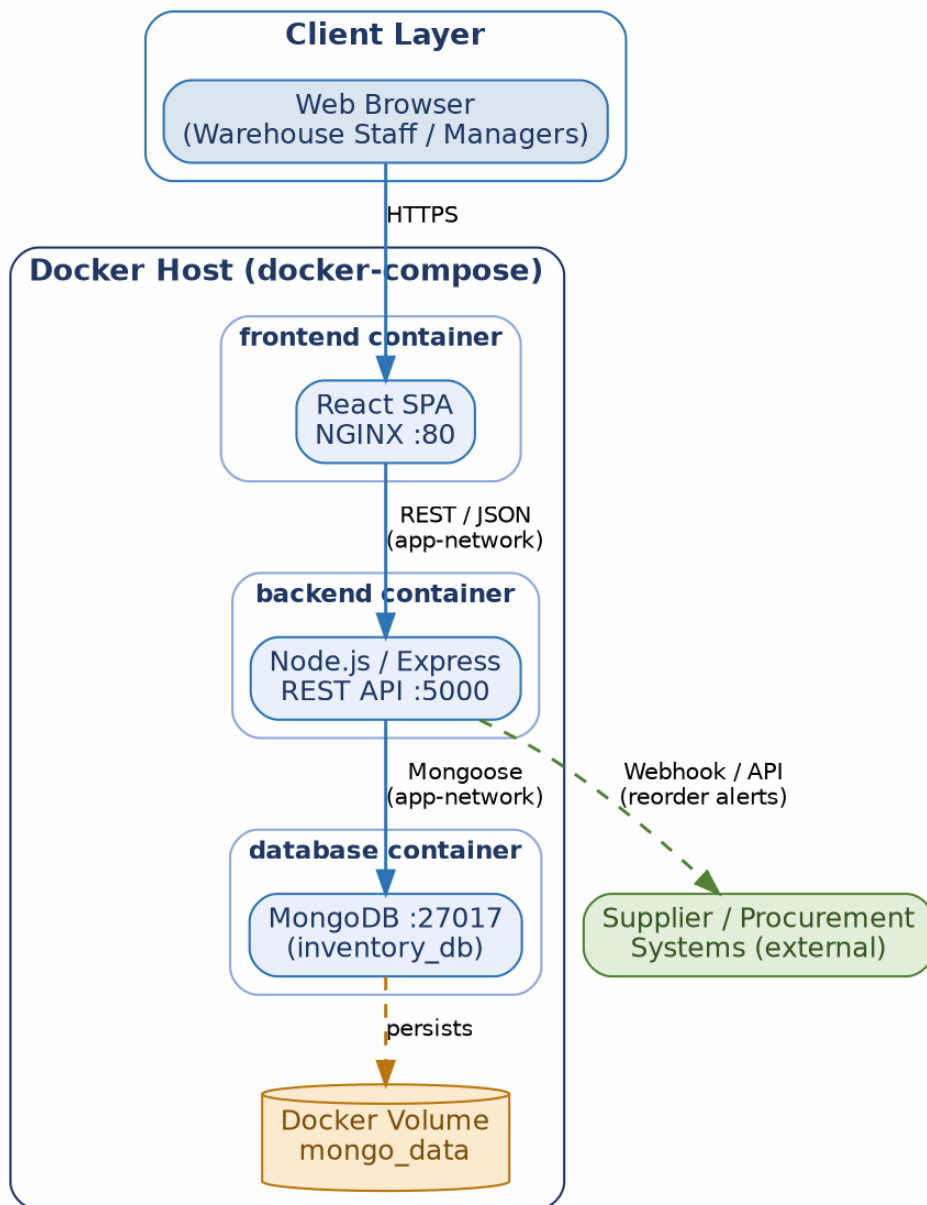


Figure 7 — Containerized system architecture for the Smart Warehouse Inventory Management Platform

8. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.
- Describe the purpose of each instruction used in the Dockerfile.

Backend Dockerfile (backend/Dockerfile)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .
ENV NODE_ENV=production
```

```
EXPOSE 5000
CMD ["node", "src/server.js"]
```

Frontend Dockerfile (frontend/Dockerfile) — multi-stage build

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Purpose of Each Instruction

Instruction	Purpose
FROM	Sets the base image (Node.js Alpine for build/runtime, NGINX Alpine to serve static files).
WORKDIR	Sets the working directory inside the container for subsequent instructions.
COPY package*.json ./	Copies only dependency manifests first so Docker can cache the npm install layer.
RUN npm ci	Installs exact dependency versions from package-lock.json for reproducible builds.
COPY . .	Copies the rest of the application source code into the image.
RUN npm run build	Compiles the React application into optimized static assets (frontend only).
FROM ... AS build / COPY --from=build	Multi-stage build: keeps only the compiled output in the final image, discarding Node build tools to reduce image size.
ENV	Sets environment variables such as <code>NODE_ENV=production</code> inside the container.
EXPOSE	Documents which port the container listens on (5000 for API, 80 for web).
CMD	Defines the default command run when the container starts.

9. Docker Compose Design

- Develop a Docker Compose configuration for the project.
- Explain the services, networking, volumes, environment variables, and dependencies used.

docker-compose.yml

```
version: "3.9"
services:
```

```

frontend:
  build: ./frontend
  ports:
    - "80:80"
  depends_on:
    - backend
  networks:
    - app-network

backend:
  build: ./backend
  ports:
    - "5000:5000"
  env_file:
    - ./backend/.env
  depends_on:
    - mongo
  networks:
    - app-network

mongo:
  image: mongo:7
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db
  networks:
    - app-network

networks:
  app-network:
    driver: bridge

volumes:
  mongo_data:

```

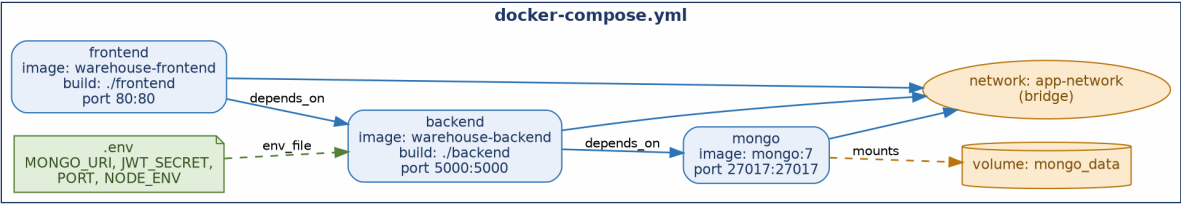


Figure 8 — Docker Compose service, network, and volume architecture

Services, Networking, Volumes, Environment Variables, and Dependencies

Element	Explanation
Services	frontend (React + NGINX), backend (Node/Express API), mongo (database) — each built and run as an independent

	container.
Networking	All three services join a shared bridge network app-network, so they can reach each other by service name (e.g., backend connects to mongo:27017).
Volumes	mongo_data is a named volume mounted at /data/db so inventory data survives container restarts and rebuilds.
Environment Variables	The backend loads MONGO_URI, JWT_SECRET, PORT, and NODE_ENV from backend/.env via env_file, keeping secrets out of the image and out of Git.
Dependencies (depends_on)	backend waits for mongo, and frontend waits for backend, so services start in the correct order.

10. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.
- Explain the testing procedure and provide evidence that the application runs successfully.

Deployment

```

bash – container deployment

$ docker compose build
[+] Building 2/2
=> [backend] naming to docker.io/library/warehouse-backend
=> [frontend] naming to docker.io/library/warehouse-frontend
$ docker compose up -d
[+] Running 4/4
✓ Network warehouse_app-network Created
✓ Volume warehouse_mongo_data Created
✓ Container warehouse-mongo Started
✓ Container warehouse-backend Started
✓ Container warehouse-frontend Started
$ docker compose ps
NAME                IMAGE                STATUS              PORTS
warehouse-frontend  warehouse-frontend   Up 8 seconds        0.0.0.0:80->80/tcp
warehouse-backend   warehouse-backend    Up 8 seconds        0.0.0.0:5000->5000/tcp
warehouse-mongo      mongo:7              Up 9 seconds        0.0.0.0:27017->27017/tcp

```

Figure 9 — Building images and starting all containers with Docker Compose

Testing Procedure and Evidence

- Health check: GET /api/health confirms the backend is running and connected to MongoDB.
- Functional test: POST /api/inventory/scan simulates a barcode scan and verifies the stock quantity updates correctly.
- Log inspection: docker compose logs backend confirms the database connection and successful request handling.
- UI verification: the React dashboard at <http://localhost> is checked in a browser to confirm it displays live stock data pulled from the API.

```
bash – deployment testing & verification

$ curl -s http://localhost:5000/api/health
{"status":"ok","service":"warehouse-backend","db":"connected"}
$ curl -s -X POST http://localhost:5000/api/inventory/scan \
-H "Content-Type: application/json" \
-d '{"sku":"WH-1042","qty":25,"binId":"A-03-12"}'
{"success":true,"sku":"WH-1042","newQuantity":25,"bin":"A-03-12"}
$ docker compose logs backend --tail 5
backend | Connected to MongoDB at mongo:27017/inventory_db
backend | Server listening on port 5000
backend | POST /api/inventory/scan 200 42ms
# Frontend verified in browser at http://localhost – dashboard loads live stock data
```

Figure 10 — API health check, functional test, and log verification

11. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

Aspect	Contribution of Git	Contribution of Docker
Collaboration	Feature branches and pull requests let multiple developers work on inventory, orders, and dashboard features in parallel without stepping on each other.	A shared docker-compose.yml means every teammate runs an identical stack, eliminating "works on my machine" issues.
Version Management	Full history, tags (v1.0.0), and diffs make it possible to trace and revert any change to the codebase.	Image tags (warehouse-backend:1.0.0) version the runtime environment alongside the code.
Portability	The repository (code + config) can be cloned on any machine with Git installed.	Containers run identically on a developer laptop, CI server, or cloud host, regardless of underlying OS.
Deployment	Merges to main represent deployable states, which can trigger automated builds.	docker compose up (or a single docker run) deploys the entire stack in minutes.
Scalability	Branching lets the team scale development effort across features independently.	Individual services (e.g., backend) can be scaled with more container replicas as warehouse load grows.
Maintainability	Clear commit history and conventional commit messages make debugging and onboarding faster.	Isolated containers make it easy to update or replace one component (e.g., upgrade MongoDB) without touching others.

12. Challenges and Improvements

- Identify the challenges encountered during implementation.
- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Challenges Encountered

- Merge conflicts arose when two feature branches modified the same route file (backend/src/routes/inventoryRoutes.js); resolved manually by reviewing both diffs before merging.
- Keeping the .env file out of Git while still documenting required variables required maintaining a separate .env.example.
- Ensuring the frontend container could reach the backend container required understanding Docker's internal DNS (service name resolution) rather than using localhost.
- Coordinating container startup order (database before backend, backend before frontend) needed depends_on plus internal retry logic, since depends_on alone does not wait for a service to be fully ready.

Future Enhancements

- Add a CI/CD pipeline (e.g., GitHub Actions) to automatically run tests and build/push Docker images on every merge to main.
- Introduce a healthcheck: block in docker-compose.yml so dependent services wait until MongoDB is actually ready, not just started.
- Adopt semantic versioning with automated changelog generation from Conventional Commits.
- Add an NGINX reverse proxy / API gateway container in front of frontend and backend for centralized TLS termination and routing.
- Introduce Kubernetes manifests for horizontal scaling of the backend service as warehouse operations grow.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation , Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5

	TOTAL	/25
--	--------------	-----